202322567

Khothalang Koetje

Phase 8- Maintainance

## 1. Change request

CR1: a lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list.

## 2. Maintenance classification

CR1 is classified as perfective maintenance because it enhances an existing workflow by adding class-wide deadline extension and student-facing visibility. The project documentation identifies the persistence table whole_class_extensions, WorkflowService class-extension logic and automated regression tests.

## 3. Impact areas

| Area | Impact/evidence to maintain |
|---|---|
| Requirements/use cases | Add class-wide extension capability and student-visible revised deadline. |
| Architecture/classes | WorkflowService API and StudentAssignment data transfer object include class extension/effective deadline information. |
| Database | whole_class_extensions persistence; verify schema and constraints. |
| Behavioural design | Update lecturer extension flow and student assignment-list flow. |
| Implementation | WorkflowService.cpp and WorkflowService.h; relevant application menu/view integration. |
| Tests | WorkflowServiceTests.cpp includes class-extension, visibility and isolation checks according to project traceability. |
| Documentation | Update manuals, traceability, release notes and regression record. |

## 4. Latest verification evidence

| Verification | Latest result | Qualification |
|---|---|---|
| Application build and launch | Passed | Developer-reported on Windows/MSYS2. |
| All registered CTest tests | Passed | Developer-reported; exact raw output/test count should be archived. |
| WorkflowService.cpp coverage | 95.38% (248/260) | gcov output supplied by developer. |
| Manual role login | All four roles confirmed | Administrator, Lecturer, Student and HoD. |
| Assignment creation/listing/submission | Confirmed for tested workflow | Developer-reported manual checks. |

The latest manual confirmations do not separately document a lecturer granting a class extension and multiple enrolled students seeing the revised deadline. That specific CR1 acceptance scenario must still be executed and recorded before CR1 is considered fully accepted.

## 5. CR1 acceptance test still required

1. Create or select a course with at least two enrolled students and one unrelated student in another course.

2. Log in as the course Lecturer and grant a class-wide extension to the assignment, with a later revised deadline and a reason.
3. Log in as each enrolled Student and verify that the assignment list shows the revised deadline/extension indicator.
4. Verify that the unrelated student does not receive the extension.
5. Grant an individual extension to one enrolled student and verify the documented precedence rule when both class and individual extensions exist.
6. Run the full CTest suite again and record the exact result, date, commit SHA and test environment.

## 6. Release readiness

| Release item | Status |
|---|---|
| CR1 implementation artefacts | Present in uploaded source/docs; inspect final UI integration. |
| Automated regression suite | Developer reports all registered tests passed. |
| Manual CR1 end-to-end acceptance | Pending specific multi-student visibility test. |
| Full regression evidence | Latest CTest result reported passed; archive raw output. |
| v1.1 release notes | Present in project docs. |
| v1.1 tag | Verify actual tag and commit in Git before claiming release. |

Conclusion: CR1 implementation and automated workflow coverage are documented, and the latest build/test report is positive. The defining CR1 user-visible acceptance check and v1.1 tag verification remain outstanding.